



INSTITUT  
POLYTECHNIQUE  
DE PARIS



ÉCOLE NATIONALE DES PONTS ET CHAUSSÉES

PROJET DE PREMIÈRE ANNÉE 2026

RAPPORT

# Réseaux de neurones pour la résolution d'équations aux dérivées partielles

Paul RIVAUD, Timoté MISSAYE, Soufiane AIT ABBOU, Pierre  
KAIRO

sous la direction de

**Sofiane EZZEHI**

Laboratoire CERMICS

# 1 Introduction

Les équations aux dérivées partielles (EDP) sont fondamentales pour modéliser de nombreux phénomènes. Traditionnellement, les EDP sont résolues à l'aide de méthodes numériques classiques telles que les méthodes des différences finies ou des éléments finis, qui reposent sur une discrétisation du domaine et de l'équation.

Dans le cadre de ce projet, nous explorons une nouvelle méthode récente et prometteuse : l'utilisation des réseaux de neurones pour approximer la solution d'une EDP. Notre étude se concentre sur deux problèmes relativement simples : l'équation de Poisson et l'équation de Helmholtz en 1D sur l'ouvert  $]0, 1[$

L'utilisation des réseaux de neurones pour cette tâche repose sur deux motivations principales :

- **La capacité d'approximation universelle** : D'un point de vue théorique, le théorème d'approximation universelle, établi notamment par Cybenko en 1989 [1], démontre que les réseaux de neurones comportant une seule couche cachée et une fonction d'activation continue (comme la tangente hyperbolique) peuvent approximer uniformément n'importe quelle fonction continue. C'est ce qui justifie mathématiquement notre choix de chercher une solution approchée dans l'espace des fonctions représentables par un réseau à une couche cachée [1].
- **L'intégration des lois de la physique** : Les approches de type PINNs permettent d'incorporer directement l'équation différentielle et les conditions aux bords dans le processus d'apprentissage. En particulier, elles peuvent être mises en œuvre dans un cadre non supervisé, en s'appuyant sur le résidu de l'EDP dans la fonction de perte, sans nécessiter de données d'entraînement étiquetées.

## 2 Problèmes test

Il existe plusieurs méthodes de résolution par réseaux de neurones pour ce problème mais elles utilisent parfois des formulations légèrement différentes. On en distingue principalement deux.

### 2.1 Formulation forte

#### 2.1.1 Equation de Poisson

Il s'agit du problème classique :

$$\begin{cases} -u''(x) = f(x) \\ u(0) = u_0 \\ u(1) = u_1 \end{cases}$$

où  $f$  est une fonction continue sur  $[0, 1]$  et  $u_0, u_1 \in \mathbb{R}$ .

Cette formulation est dite forte car on cherche a priori la solution sur  $C^2(\Omega)$  et comme nous le verrons par la suite, la résolution va nécessiter une double dérivation.

### 2.1.2 Equation de Helmholtz modifié

C'est le problème différentiel suivant :

$$-\Delta u + \lambda u = f \text{ sur } \Omega \quad (1)$$

avec  $0 < \lambda$  et  $f \in L^2(\Omega)$

## 2.2 Formulation faible (variationnelle)

### 2.2.1 Equation de Poisson

Dans notre cas où  $\Omega = (0, 1)$  la formulation faible est : trouver  $u \in H_0^1(0, 1)$  tel que

$$\forall v \in H_0^1(0, 1), \quad a(u, v) = b(v)$$

avec la forme bilinéaire :

$$a(u, v) = \int_0^1 u'(x) v'(x) dx$$

et la forme linéaire :

$$b(v) = \int_0^1 f(x) v(x) dx$$

Elle est dite faible car on cherche une solution dans l'espace  $H_0^1(0, 1)$  plus permissif et car nous verrons dans la partie suivante que sa résolution ne nécessitera qu'une dérivation simple. Une démonstration de la formule faible est fournie dans l'annexe.

### 2.2.2 Equation de Helmholtz modifiée

La formule variationnelle pour cet équation s'écrit :

$$\forall v \in H_0^1(\Omega) : \int_{\Omega} \nabla u \cdot \nabla v dx + \lambda \int_{\Omega} uv dx = \int_{\Omega} f v dx \quad (2)$$

Elle peut être également écrite sous la forme précédente avec  $a(u, v) = \int_{\Omega} (\nabla u \cdot \nabla v + \lambda uv) dx$  et  $b(v) = \int_{\Omega} f v dx$ . (On garde la notation du gradient ; en 1D, il s'agit simplement de la dérivée)

## 3 Méthodes de résolution

Un réseau de neurones artificiel (Artificial Neural Network, ANN) est un modèle de calcul paramétrique constitué d'un graphe orienté de nœuds (neurones) organisés en couches, dans lequel chaque neurone calcule une transformation non-linéaire d'une combinaison linéaire pondérée de ses entrées [fig.1]. La transformation est rendue non-linéaire par l'intermédiaire d'une fonction

d'activation. Un réseau profond (Deep Neural Network) empile plusieurs couches de transformations successives. L'apprentissage consiste à minimiser une fonction de perte sur un ensemble de données d'entraînement, par descente de gradient stochastique (SGD) et rétropropagation du gradient (backpropagation).

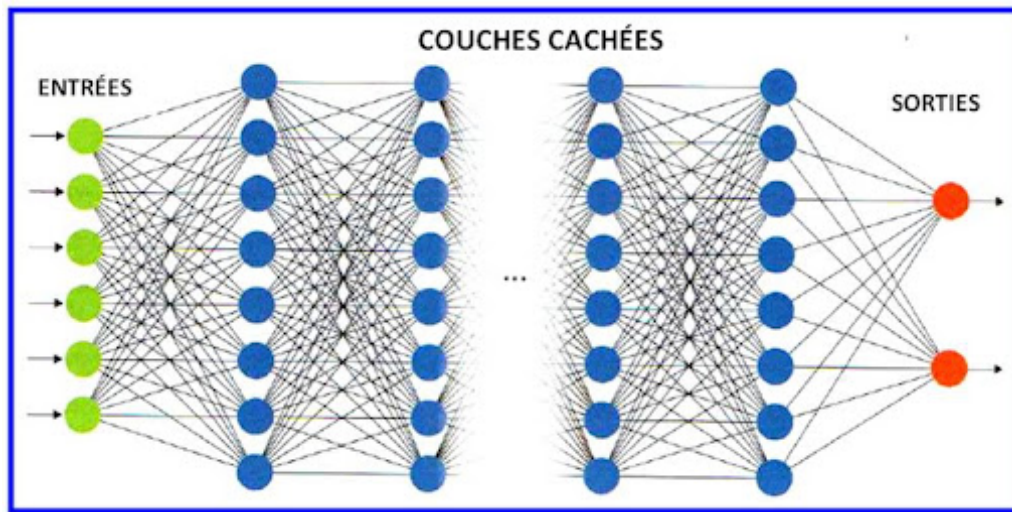


FIGURE 1 – Architecture d'un réseau de neurones

L'idée d'utiliser des algorithmes d'apprentissage automatique pour résoudre des problèmes physiques a connu un essor fulgurant ces dernières années. Dans le contexte de la résolution d'EDP, deux paradigmes majeurs se distinguent dans la littérature récente et fondent les bases de notre approche :

### 3.1 Méthode PINNs (Physics-Informed Neural Networks) :

Introduite par Raissi, Perdikaris et Karniadakis [3], cette approche repose sur l'utilisation de la différenciation automatique pour calculer les dérivées du réseau de neurones par rapport aux variables d'entrée. Contrairement aux méthodes variationnelles de type Deep Ritz, qui consistent à minimiser une fonctionnelle d'énergie issue d'une formulation faible du problème, les PINNs s'appuient sur la formulation forte de l'équation différentielle.

Plus précisément, le résidu de l'EDP est directement intégré dans la fonction de perte, auquel on ajoute des termes de pénalisation des conditions aux bords. L'apprentissage du réseau consiste alors à minimiser cette fonction de perte, ce qui permet d'approximer une solution de l'EDP.

Ce cadre peut être utilisé en l'absence de données d'entraînement étiquetées, et s'applique aussi bien à des problèmes directs qu'à des problèmes inverses.

Plus en détail : on cherche l'ensemble solution sur un espace fonctionnel adapté  $V$  (par exemple  $V \subset H^1(\Omega)$ )

$$\begin{cases} -u''(x) = f(x) \\ u(0) = u_0 \\ u(1) = u_1 \end{cases}$$

où  $f$  est une fonction continue sur  $[0, 1]$  et  $u_0, u_1 \in \mathbb{R}$ .

On définit la fonctionnelle d'énergie que l'on note  $E$  :

$$E(u) = \int_{\Omega} |u''(x) + f(x)|^2 dx + (u(0) - u_0)^2 + (u(1) - u_1)^2$$

La fonctionnelle  $E(u)$  mesure à la fois le résidu intérieur, c'est-à-dire à quel point  $u$  satisfait l'équation différentielle sur le domaine et le résidu aux bords, qui pénalise toute violation des conditions aux limites. Cette formulation transforme le problème différentiel en un problème d'optimisation.

Ainsi, si  $u^*$  est la solution exacte :  $u^*$  vérifie les conditions aux limites et pour tout  $x \in \Omega$ ,  $f(x) + (u^*)''(x) = 0$  Alors

$$E(u^*) = 0.$$

On définit l'espace des réseaux de neurones :

$$V_{nn} = \left\{ f : x \mapsto \sum_{i=1}^n c_i \sigma(w_i x + b_i) \mid \theta = (c_i, w_i, b_i)_{1 \leq i \leq n} \in \mathbb{R}^{3n} \right\}$$

où  $\sigma$  désigne la fonction d'activation

Pour résumé, les solutions sont définies comme :

— Solution exacte :

$$u^* = \arg \min_{v \in V} E(v)$$

— Solution réseau de neurones :

$$u_{nn} = \arg \min_{v \in V_{nn}} E(v)$$

Cela revient à trouver :

$$\theta^* = \arg \min_{\theta \in \mathbb{R}^{3n}} E(u_{\theta})$$

La recherche de  $\theta^*$  est réalisée par descente de gradient, en calculant le gradient de  $E(u_{\theta})$  par rapport à  $\theta$  via la différentiation automatique.

## 3.2 Méthode Deep Ritz

Proposée par E et Yu (2017)[2], cette méthode adapte le principe de la méthode de Ritz aux architectures d'apprentissage profond. Au lieu de chercher à résoudre l'équation de manière directe, le problème est reformulé sous la forme d'un problème variationnel visant à minimiser les paramètres de notre fonction. Cela conduit à une fonctionnelle à minimiser légèrement différente que celle de la méthode PINN, son établissement est détaillé dans la partie suivante. Le réseau de neurones est utilisé pour construire la fonction d'activation, et l'optimisation s'effectue naturellement grâce à des algorithmes de descente de gradient.

### 3.2.1 Fonctionnelle de l'énergie

Dans le cas où  $a$  est de plus symétrique, alors la solution unique du problème précédent est aussi l'unique solution qui vérifie :

$$J(u) = \inf_{v \in H} J(v)$$

où  $J(v) = \frac{1}{2}a(v, v) - b(v)$ .

C'est l'énergie qu'on va minimiser dans notre implémentation de la méthode, bien sûr en s'assurant que  $a$  du problème étudié est symétrique.

Une démonstration de ce résultat est dans l'annexe.

### 3.2.2 Fonctionnelle de l'énergie pour l'équation de Poisson :

Le fonctionnelle de l'énergie pour l'équation de Poisson est :

$$J(u) = \frac{1}{2} \int_{\Omega} |\nabla u|^2 dx - \int_{\Omega} f u dx$$

Le lecteur peut voir que la forme bilinéaire de la formule variationnelle de ce problème est symétrique. De même pour l'autre équation.

### 3.2.3 Fonctionnelle de l'énergie pour le problème de Helmholtz modifié

Il s'écrit sous la forme :

$$J(u) = \frac{1}{2} \int_{\Omega} |\nabla u|^2 dx + \frac{1}{2} \int_{\Omega} |u|^2 dx - \int_{\Omega} f u dx$$

On verra dans le détail d'implémentation que ces énergies seront modifiées pour respecter aussi les conditions aux limites.

### 3.3 Détail d'implémentation

#### 3.3.1 Equation de Poisson

Rappelons que l'on cherche l'ensemble solution sur un espace fonctionnel adapté  $V$  de :

$$\begin{cases} -u''(x) = f(x) \\ u(0) = u_0 \\ u(1) = u_1 \end{cases}$$

où  $f$  est une fonction continue sur  $[0, 1]$  et  $u_0, u_1 \in \mathbb{R}$ .

Nous avons d'abord implémenté la méthode PINN avec un réseau de neurone d'une seule couche, à  $n$  neurones. On pose  $n$  le nombre de neurones on a donc  $3n$  paramètres.

Dans ce cadre la fonctionnelle d'énergie est :

$$E(u) = \int_{\Omega} |u''(x) + f(x)|^2 dx + (u(0) - u_0)^2 + (u(1) - u_1)^2$$

En pratique, l'intégrale continue est approchée par une règle des rectangles sur  $N$  points équirépartis :

$$\Delta x \sum_{i=1}^N \left( u''(x_i) + f(x_i) \right)^2 + \left( u(0) - u_0 \right)^2 + \left( u(1) - u_1 \right)^2$$

avec

$$x_i = i\Delta x, \quad i = 1, \dots, N$$

Pour la méthode Deep Ritz, la fonctionnelle est :

$$\mathcal{J}(u) = \int_0^1 \frac{1}{2} |u'(x)|^2 - f(x)u(x) dx$$

Que nous implémentons également avec la méthode des rectangles comme suit :

$$\mathcal{J}(\theta) = \frac{1}{N} \sum_{i=1}^N \left[ \frac{1}{2} \left( u'_{\theta}(x_i) \right)^2 - f(x_i) u_{\theta}(x_i) \right] + \alpha u_{\theta}(0)^2 + \beta u_{\theta}(1)^2$$

Notez que nous avons ajouté une pénalisation au bord pondérée par des coefficients réels  $\alpha$  et  $\beta$ . L'influence de la pénalisation sur les résultats sera mise en évidence dans le problème de Helmholtz.

En premier lieu la fonction d'activation choisie était :

$$\sigma(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

La fonction  $\tanh$  est un choix naturel pour ce problème car elle est deux fois continûment dérivable sur  $\mathbb{R}$ , ce qui garantit que les éléments de  $V_{nn}$  appartiennent bien à  $C^2(\Omega)$  et que  $u''_\theta$  est calculable analytiquement. Ses dérivées s'expriment simplement :  $\sigma'(x) = 1 - \tanh^2(x)$ , ce qui facilite le calcul du gradient lors de l'optimisation.

Nous avons ensuite essayé d'autres fonctions d'activation, ceci est détaillé dans la partie sur les hyperparamètres.

Enfin pour trouver

$$\theta^* = \arg \min_{\theta \in \mathbb{R}^{3n}} E(u_\theta)$$

Nous avons procédé par différentiation automatique Adam.

### 3.3.2 Equation de Helmholtz

Pour ce problème, on n'applique que la méthode DeepRitz où on minimise la fonction suivante :

$$\mathcal{J}(\theta) = \frac{1}{N} \sum_{i=1}^N \left[ \frac{1}{2} (u'_\theta(x_i))^2 + \frac{1}{2} (u_\theta(x_i))^2 - f(x_i) u_\theta(x_i) \right] + \alpha u_\theta(0)^2 + \beta u_\theta(1)^2$$

On observera la convergence vers la solution et l'effet de pénalisation. La fonction d'activation utilisée est  $\tanh$  et l'optimiseur est Adam avec un taux d'apprentissage de 0.1.

## 4 Résultats numériques

Cette section présente l'ensemble des résultats expérimentaux obtenus sur l'équation de Poisson 1D. Nous suivons la progression du notebook : premier test PINN, étude de la fonction d'activation et des hyperparamètres, architecture multi-couches, puis comparaison avec la méthode Deep Ritz.

### 4.1 PINN à une couche cachée — premier test

#### Mise en place

On cherche à approcher la solution de

$$-u''(x) = f(x), \quad x \in ]0, 1[, \quad u(0) = u(1) = 0,$$

avec  $f(x) = \sin(2\pi x)$ , de solution exacte  $u^*(x) = \sin(2\pi x)/(4\pi^2)$ .

Le réseau `snn` possède une unique couche cachée de  $n = 3$  neurones (activation `tanh`), soit



$3n = 9$  paramètres. La fonctionnelle PINN

$$\mathcal{E}(u_\theta) = \frac{1}{N} \sum_{i=1}^N \left( u_\theta''(x_i) + f(x_i) \right)^2 + u_\theta(0)^2 + u_\theta(1)^2$$

est minimisée par **Adam** ( $\eta = 0,01$ , 1000 époques,  $N = 1000$  points de collocation équirépartis).

## Évolution au cours de l'entraînement

Convergence progressive du réseau PINN (3 neurones, **tanh**) vers la solution exacte  $u^*$  (en bleu) au fil de l'entraînement. À l'époque 0, la sortie du réseau (en orange) est une droite issue de l'initialisation aléatoire des poids, sans lien avec la solution. À l'époque 300, la forme sinusoïdale est identifiée mais l'amplitude est sous-estimée d'environ 40 % et les conditions aux bords ne sont pas encore satisfaites. À l'époque 900, les deux courbes se superposent presque sur  $[0,4; 1,0]$  ; un léger écart subsiste en  $x = 0$  et  $x = 1$ , révélant que les termes de pénalisation des conditions de Dirichlet ne sont pas encore annulés à cet horizon d'entraînement.

## Convergence de la loss

Ces résultats valident l'implémentation : l'optimiseur Adam, couplé à la différenciation automatique (`torch.func.grad + vmap`), minimise correctement la fonctionnelle PINN. La loss finale de  $5 \times 10^{-3}$  pour 3 neurones seulement est encourageante et motive l'étude systématique des hyperparamètres.

## 4.2 Étude de la fonction d'activation et des hyperparamètres

### Fonctions d'activation comparées

#### Heatmap de la perte de validation

Nous avons balayé  $n \in \{5, 10, 20, 50\}$  neurones et les trois activations ( $N_{\text{train}} = N_{\text{val}} = 1000$ , 1000 époques), en conservant la perte de validation minimale atteinte durant l'entraînement.

Ces résultats conduisent aux conclusions suivantes :

1. **tanh est le meilleur choix** d'activation pour les PINNs résolus en formulation forte, grâce à sa dérivabilité à tous les ordres.
2. **10 neurones suffisent** pour **tanh** sur ce problème 1D : le gain en passant à 50 neurones est inférieur à 2 %.
3. **ReLU est inutilisable** dans ce cadre, quelle que soit la taille du réseau.

## 4.3 Architecture multi-couches — impact de la profondeur

Nous avons testé la classe NN pour  $L \in \{3, 5, 7, 10, 13, 17, 20\}$  couches cachées, avec  $n_{\text{neurones}} = 5$  et l'activation **tanh**.

Epoch 0, Loss: 1.0090153217315674

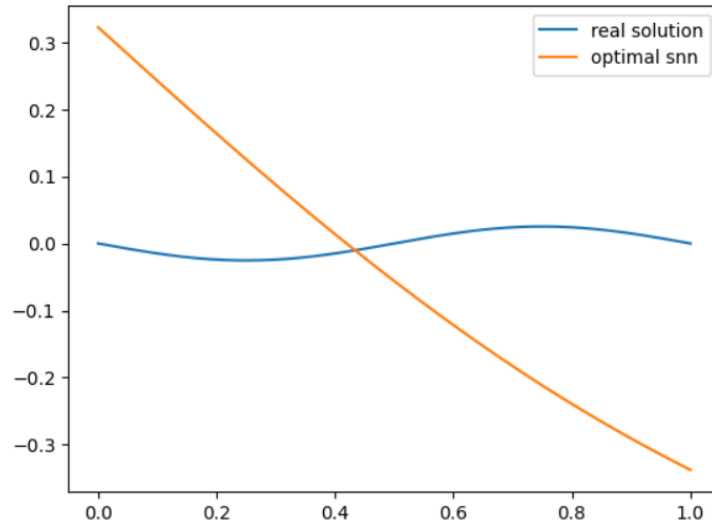


FIGURE 2 – epoch 0

Epoch 300, Loss: 0.012232286855578423

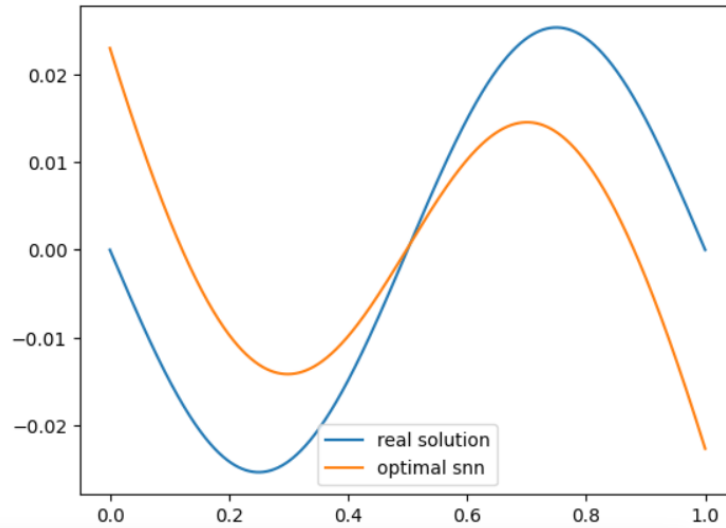


FIGURE 3 – epoch 300

Epoch 900, Loss: 0.005132877733558416

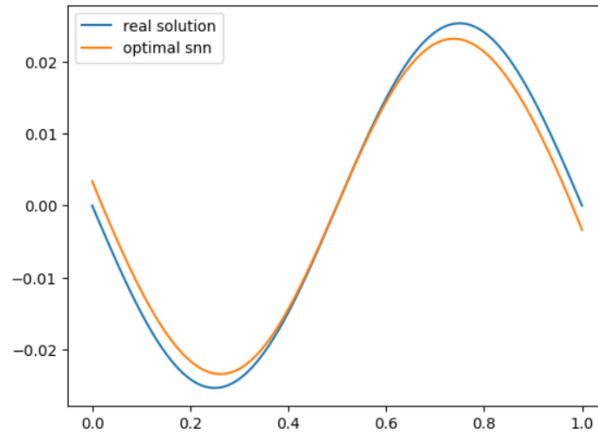


FIGURE 4 – epoch 900

FIGURE 5 – Convergence progressive du réseau PINN

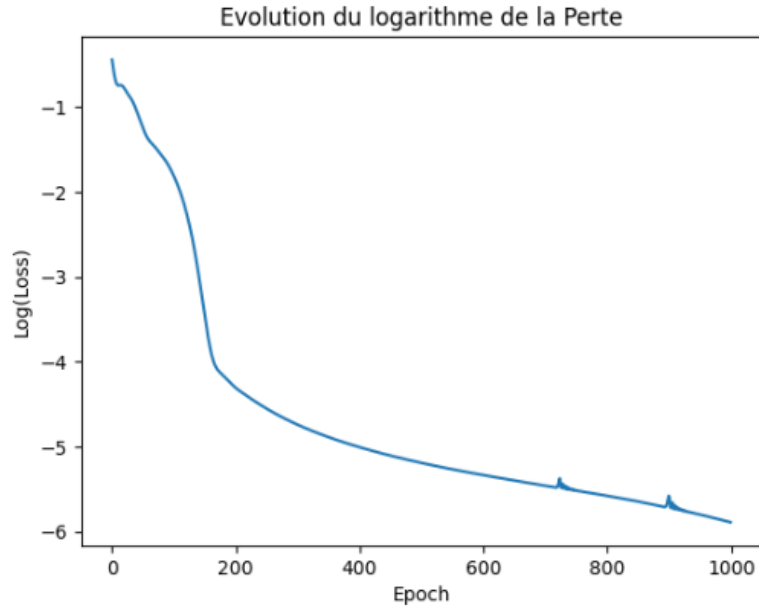


FIGURE 6 – Évolution du logarithme de perte  $\mathcal{E}(u_\theta)$  en fonction de l'époque (PINN, 3 neurones,  $\tanh$ ). La loss chute très rapidement de 1,0 à  $\approx 0,05$  durant les 150 premières époques : c'est la phase de *descente rapide* où le réseau apprend la forme globale de la solution. Elle se stabilise ensuite autour de  $5 \times 10^{-3}$  à partir de l'époque 250, formant un plateau caractéristique de la saturation capacitaire : avec seulement 3 neurones,  $V_{nn}$  n'est pas assez riche pour annuler complètement le résidu de l'EDP.

**Conclusion pratique :** pour l'équation de Poisson 1D, une architecture peu profonde ( $L \leq 5$  couches) avec  $n \geq 10$  neurones est nettement préférable à un réseau très profond.

#### 4.4 Synthèse des résultats

| Configuration | Activation       | Neurones | Couches | Loss val. min.               |
|---------------|------------------|----------|---------|------------------------------|
| PINN de base  | $\tanh$          | 3        | 1       | $5,13 \times 10^{-3}$        |
| PINN optimisé | $\tanh$          | 5        | 1       | $1,03 \times 10^{-4}$        |
| PINN optimisé | $\tanh$          | 10       | 1       | $1,71 \times 10^{-5}$        |
| PINN optimisé | $\tanh$          | 20       | 1       | $1,67 \times 10^{-5}$        |
| PINN optimisé | $\tanh$          | 50       | 1       | $1,80 \times 10^{-5}$        |
| PINN optimisé | $\text{sigmoid}$ | 50       | 1       | $1,58 \times 10^{-5}$        |
| PINN optimisé | $\text{ReLU}$    | 50       | 1       | $4,99 \times 10^{-1}$        |
| DNN stable    | $\tanh$          | 5        | 13      | $\approx 10^{-2}$            |
| DNN instable  | $\tanh$          | 5        | 20      | $\approx 4,8 \times 10^{-1}$ |

TABLE 1 – Récapitulatif des performances pour les différentes configurations testées sur l'équation de Poisson 1D.

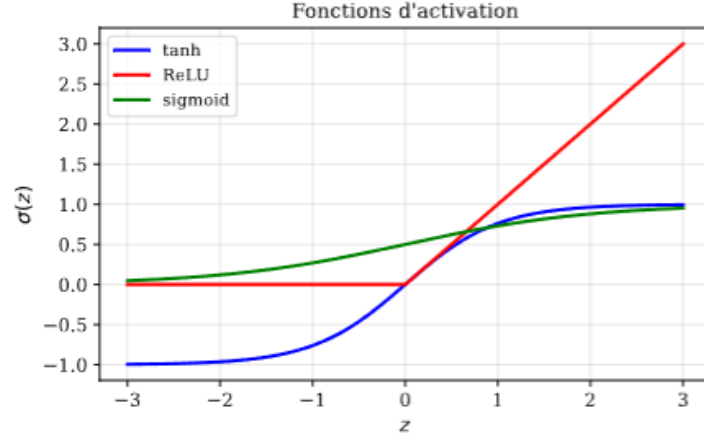


FIGURE 7 – Graphe des trois fonctions d’activation testées : **tanh** (bleu), **ReLU** (rouge) et **sigmoid** (vert). **tanh** et la fonction sigmoïde sont infiniment dérivable : une propriétés essentielles pour que la dérivée seconde  $u''_\theta$ , calculée par différenciation automatique, soit définie et non-nulle en tout point du domaine. **ReLU** a une dérivée seconde nulle presque partout, ce qui rend le résidu  $u''_\theta(x_i) + f(x_i)$  structurellement non-minimisable. mais sature rapidement vers 1 pour  $z > 2$ , ce qui ralentit la convergence par le phénomène de vanishing gradient.

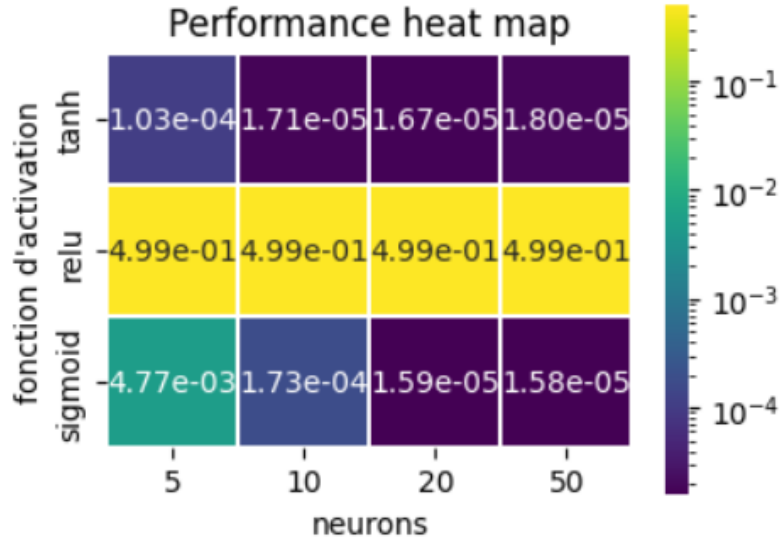


FIGURE 8 – Heatmap (échelle logarithmique) de la perte de validation minimale en fonction de la fonction d’activation (axe vertical) et du nombre de neurones (axe horizontal). Les cases sombres (violet) correspondent aux meilleures performances. **tanh** obtient des losses de  $\approx 10^{-4}$ – $10^{-5}$  dès 10 neurones, et le gain au-delà est marginal ( $1,71 \times 10^{-5}$  à 10 contre  $1,67 \times 10^{-5}$  à 20 et  $1,80 \times 10^{-5}$  à 50). **ReLU** échoue complètement ( $\approx 5 \times 10^{-1}$  dans toutes les configurations), ce qui confirme son inadéquation pour les PINNs en formulation forte : sa dérivée seconde nulle p.p. empêche la minimisation du résidu  $u''_\theta$ . **sigmoid** converge correctement mais reste un ordre de grandeur au-dessus de **tanh** pour 5 neurones ( $4,77 \times 10^{-3}$ ) avant de rattraper ses performances pour 20–50 neurones ( $\approx 1,6 \times 10^{-5}$ ).

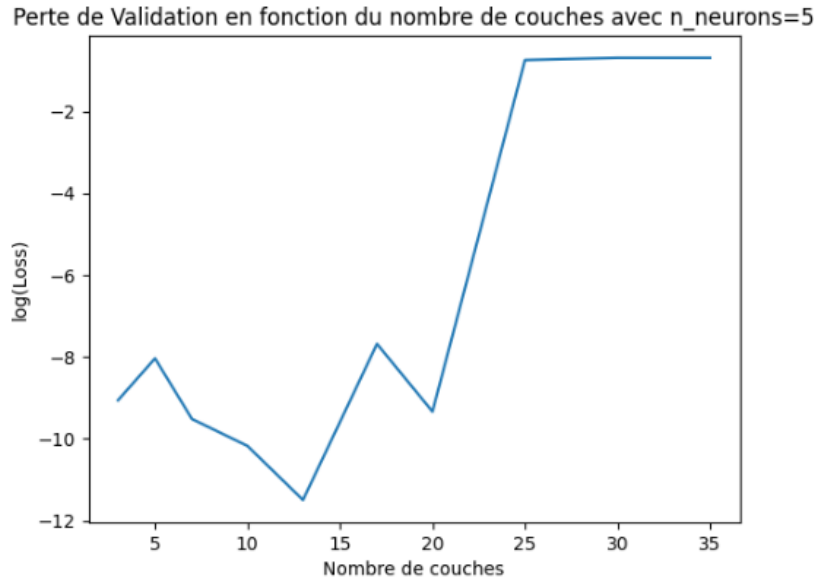


FIGURE 9 – Logarithme de la perte de validation en fonction du nombre de couches cachées ( $n_{\text{neurons}} = 5$ , `tanh`, 1000 époques). La perte reste faible et quasi-stable pour  $L \leq 13$  (de l'ordre de  $10^{-3}$ – $10^{-2}$ ) : la profondeur supplémentaire n'apporte rien car le problème 1D n'a pas de structure hiérarchique qui la justifierait. À partir de  $L = 17$ , la perte explose brusquement pour atteindre  $\approx 0,48$  à  $L = 20$ , soit une dégradation d'un facteur 50. Ce saut brutal est la signature du *vanishing gradient* : dans les couches profondes avec `tanh`, les gradients sont multipliés par  $1 - \tanh^2(z) \approx 0$ , rendant la rétropropagation inefficace et l'entraînement instable.

## 4.5 Méthode DeepRitz

### 4.5.1 Résultats numériques pour l'équation de Poisson

On prend  $f(x) = \sin(2\pi x)$ . On effectue un entraînement de 3000 epochs en prenant  $\alpha = \beta = 10$ , pour chaque 200 itérations on affiche la courbe obtenue, la valeur du fonctionnelle de l'énergie ( *loss* ) et la valeur de la norme  $L^2$  de la différence entre la solution réelle et la fonction du réseau de neurones. (Loss L2)

### 4.5.2 Comparaison

Même avec 3000 epochs, l'algorithme DeepRitz n'a pas assez convergé vers la solution relativement à PINN. Certes, ce dernier s'avère plus efficace, mais DeepRitz est d'abord plus économique au niveau des calculs ( on n'a pas dérivé 2 fois), et plus facile à appliquer comme pour le problème suivant.

Loss L2: 0.5652156472206116

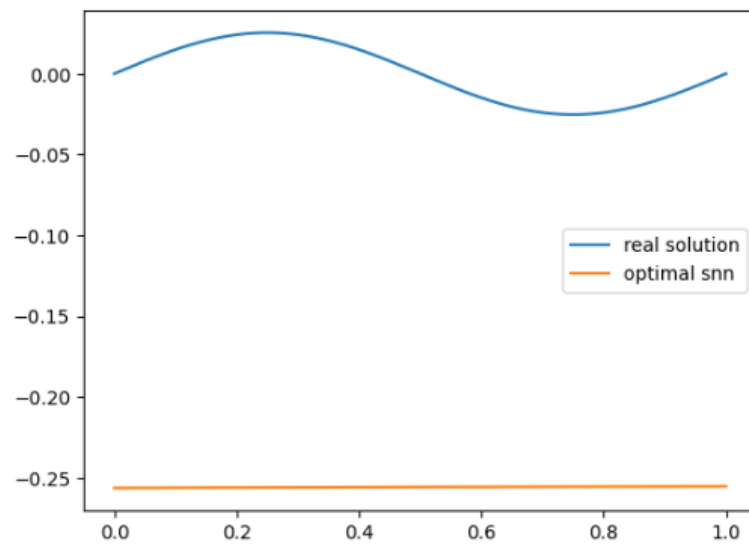


FIGURE 10 – epoch 0

Epoch 400, Loss: -0.005836250726133585

Loss L2: 0.47799134254455566

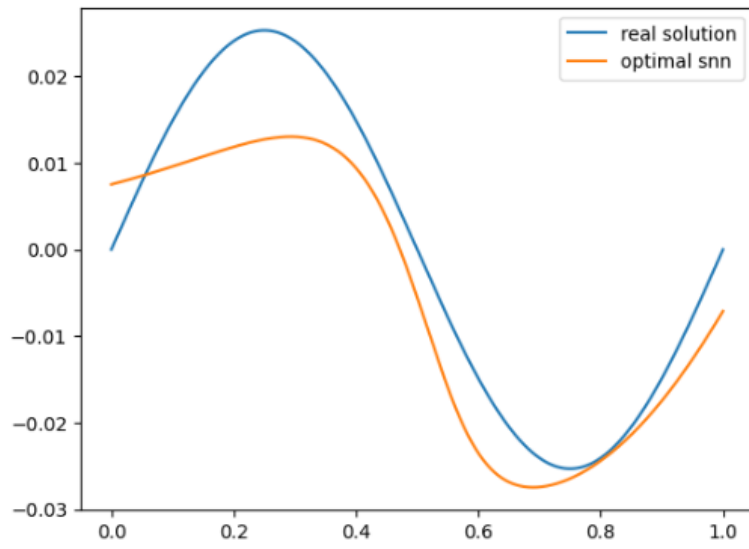


FIGURE 11 – epoch 400

Epoch 2400, Loss: -0.007369029801338911

Loss L2: 0.47110891342163086

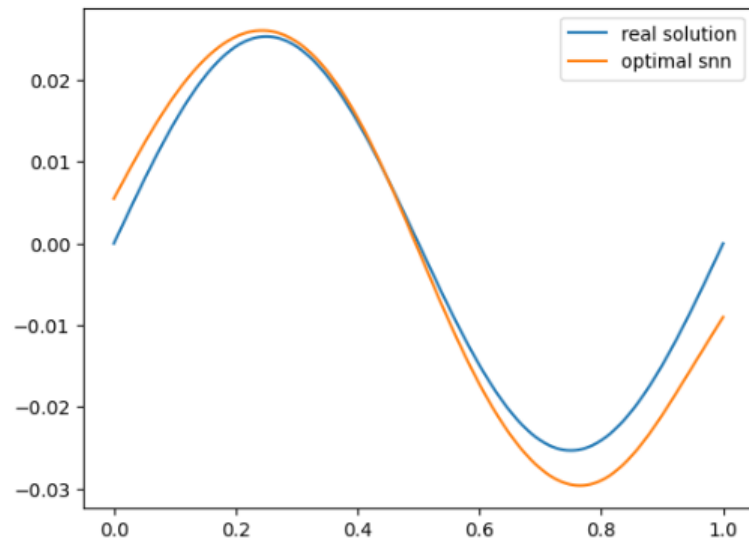


FIGURE 12 – epoch 2400

FIGURE 13 – Convergence progressive du réseau DeepRitz appliqué au problème de Poisson

## 4.6 Résultats numériques pour l'équation de Helmholtz

On fait 1000 itérations (epochs) en prenant pour premier essai  $\alpha = \beta = 10$  et  $\lambda = 1$  (ainsi la solution du problème est  $u(x) = \frac{-\sin(2\pi x)}{1+4\pi^2}$ ). On remarque que cette fois ci on a une convergence plus rapide. (LossL2 de l'ordre de  $10^{-5}$  après même pas 1000 itérations).

### 4.6.1 Hyperparamètre : les facteurs de pénalisation $\alpha$ et $\beta$

On affiche pour différentes valeurs de  $\alpha$  et  $\beta$  ( en gardant  $\alpha = \beta$  ) la courbe obtenue après l'entraînement (epochs=1000,  $\lambda = 1$ ).



Epoch 200, Loss: -0.0030263937078416348  
 Loss L2: 6.254143227124587e-05

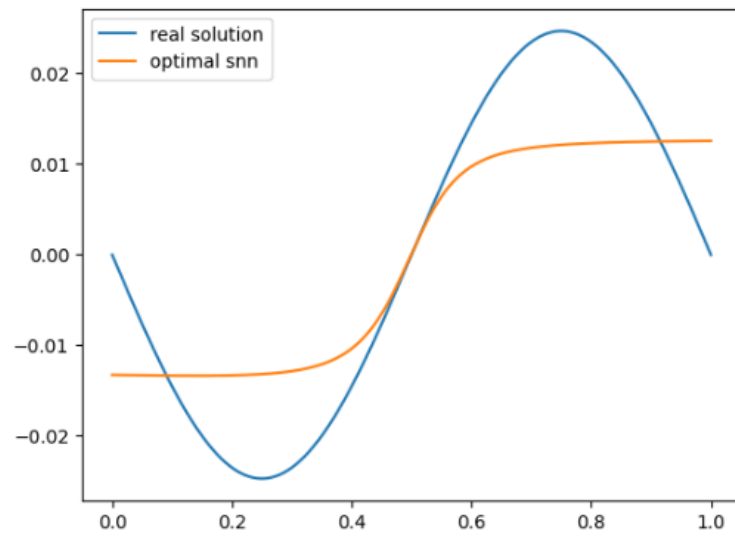


FIGURE 14 – epoch 200

Epoch 400, Loss: -0.0072404746897518635  
 Loss L2: 1.532888381916564e-05

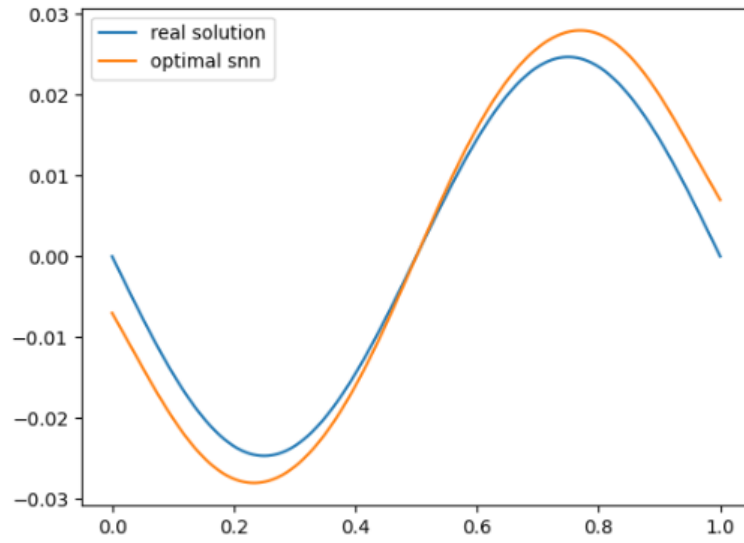


FIGURE 15 – epoch 400

Epoch 900, Loss: -0.007243327330797911  
 Loss L2: 1.5429744962602854e-05

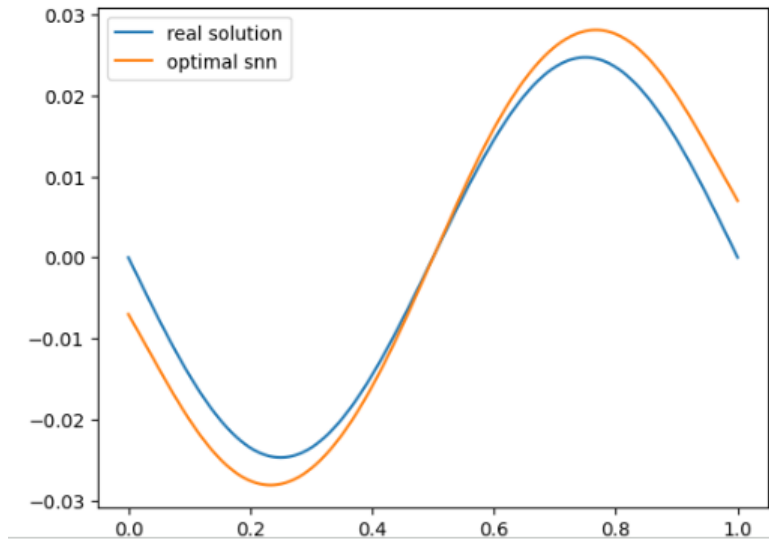


FIGURE 16 – epoch 900

FIGURE 17 – Deep Ritz appliqué à l'équation de Helmholtz

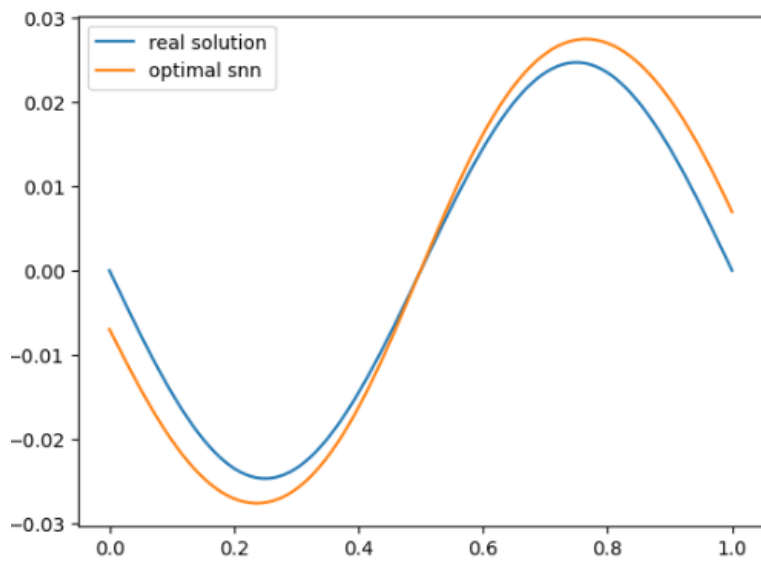


FIGURE 18 –  $\alpha = \beta = 10$

Epoch 900, Loss: -0.006244342308491468  
Loss L2: 2.2746370120785286e-07

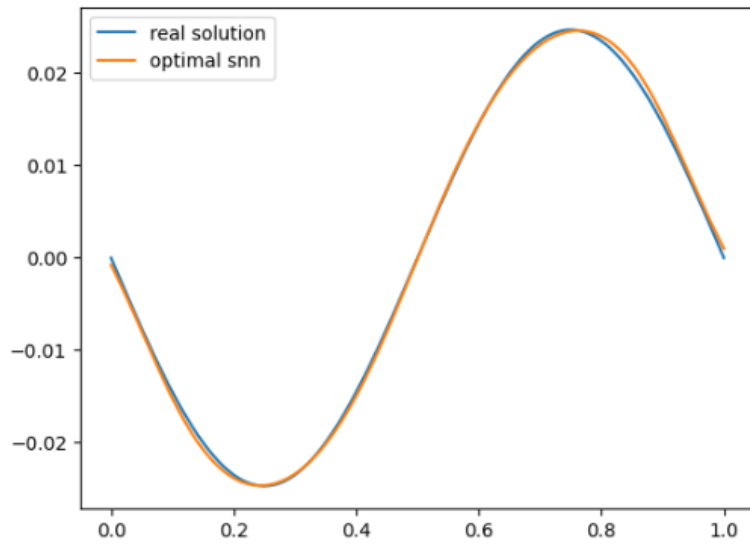


FIGURE 19 –  $\alpha = \beta = 100$

Epoch 900, Loss: -0.0004608966992236674  
Loss L2: 0.0002731589484028518

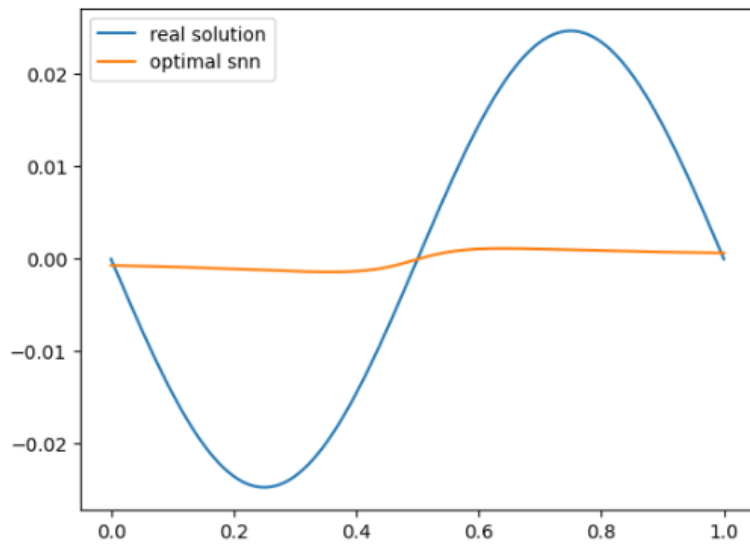


FIGURE 20 –  $\alpha = \beta = 200$

FIGURE 21 – Solution approchée pour différentes valeurs de  $\alpha$  et  $\beta$

### 4.6.2 Commentaire

Ce problème constitue un point de différence intéressant entre la méthode PINN et DeepRitz : introduire une énergie dans la méthode PINN dans ce problème n'est pas évident. La méthode DeepRitz nous permet d'établir une fonctionnelle sans avoir besoin même de calculer les dérivées secondes. Même si la convergence est lente relativement, ça reste une méthode plus pratique que PINN dans certains cas.

Pour l'effet des paramètres de pénalisations, on remarque qu'une pénalisation très faible engendre un "underfitting", c'est-à-dire que l'algorithme ne considère pas suffisamment les conditions aux limites. Par contre, une pénalisation très importante engendre le "surfitting", l'algorithme pose en priorité les conditions limites et "oublie" l'équation sur l'ouvert.

## A Rappels et résultats théoriques

### A.1 Les espaces de Sobolev

$H^1(\Omega)$  :

Soit  $\Omega$  un ouvert de  $\mathbb{R}^n$ . L'espace de Sobolev  $H^1(\Omega)$  est l'ensemble des fonctions de  $L^2(\Omega)$  dont les dérivées partielles premières au sens des distributions appartiennent également à  $L^2(\Omega)$  :

$$H^1(\Omega) = \left\{ u \in L^2(\Omega) \mid \frac{\partial u}{\partial x_i} \in L^2(\Omega), \quad \forall i \in \{1, \dots, n\} \right\}$$

Cet espace est muni de la norme :

$$\|u\|_{H^1(\Omega)} = \left( \|u\|_{L^2(\Omega)}^2 + \|\nabla u\|_{L^2(\Omega)}^2 \right)^{\frac{1}{2}}$$

Définition de l'espace  $H_0^1(\Omega)$  :

L'espace  $H_0^1(\Omega)$  est défini comme l'adhérence de l'espace des fonctions tests  $\mathcal{D}(\Omega)$  (fonctions de classe  $C^\infty$  à support compact inclus dans  $\Omega$ ) pour la norme de  $H^1(\Omega)$  :

$$H_0^1(\Omega) = \overline{\mathcal{D}(\Omega)}^{\|\cdot\|_{H^1}}$$

Pour un domaine  $\Omega$  suffisamment régulier, cela correspond aux fonctions de  $H^1(\Omega)$  ayant une trace nulle sur la frontière  $\partial\Omega$  (c'est-à-dire  $u = 0$  sur  $\partial\Omega$ ).

### A.2 Théorème de Lax-Milgram

Soit  $V$  un espace de Hilbert réel muni de la norme  $\|\cdot\|_V$ . On considère :

— Une forme bilinéaire  $a(\cdot, \cdot) : V \times V \rightarrow \mathbb{R}$ , continue.

— Une forme linéaire  $L(\cdot) : V \rightarrow \mathbb{R}$ , continue.

Le problème variationnel consiste à :

Trouver  $u \in V$  tel que :

$$a(u, v) = L(v) \quad \forall v \in V \quad (3)$$

Le théorème de Lax-Milgram montre que si la forme bilinéaire  $a$  est coercive sur  $H_0^1$  alors ce problème admet une unique solution.

### A.3 Démonstration du passage à la formule variationnelle :

On part du problème où l'on cherche  $u \in H_0^1(\Omega)$  telle que :

$$-\Delta u = f \quad \text{dans } \mathcal{D}'(\Omega)$$

avec  $f \in L^2(\Omega)$ . L'égalité  $-\Delta u = f$  au sens des distributions signifie que pour toute fonction test  $\varphi \in \mathcal{D}(\Omega)$ , on a :

$$\langle -\Delta u, \varphi \rangle_{\mathcal{D}', \mathcal{D}} = \langle f, \varphi \rangle_{\mathcal{D}', \mathcal{D}}$$

Comme  $f \in L^2(\Omega)$ , elle est localement intégrable, donc l'action de la distribution  $f$  sur  $\varphi$  coïncide avec l'intégrale classique de Lebesgue :

$$\langle f, \varphi \rangle_{\mathcal{D}', \mathcal{D}} = \int_{\Omega} f \varphi \, dx$$

De plus, une intégration par partie au sens des distributions donne :

$$\langle -\Delta u, \varphi \rangle_{\mathcal{D}', \mathcal{D}} = \left\langle -\sum_{i=1}^n \frac{\partial^2 u}{\partial x_i^2}, \varphi \right\rangle_{\mathcal{D}', \mathcal{D}} = \sum_{i=1}^n \left\langle \frac{\partial u}{\partial x_i}, \frac{\partial \varphi}{\partial x_i} \right\rangle_{\mathcal{D}', \mathcal{D}}$$

ainsi pour toute fonction test  $\varphi \in \mathcal{D}(\Omega)$  :

$$\int_{\Omega} \nabla u \cdot \nabla \varphi \, dx = \int_{\Omega} f \varphi \, dx$$

on définit la forme bilinéaire donnée par :

$$a(u, v) = \int_{\Omega} \nabla u \cdot \nabla v \, dx$$

Elle est continue car :

$$|a(u, v)| = \left| \int_{\Omega} \nabla u \cdot \nabla v \, dx \right| \leq \|\nabla u\|_{L^2(\Omega)} \|\nabla v\|_{L^2(\Omega)}$$

Or, par définition de la norme dans  $H^1(\Omega)$ , on sait que :

$$\|u\|_{H^1(\Omega)} = \left( \|u\|_{L^2(\Omega)}^2 + \|\nabla u\|_{L^2(\Omega)}^2 \right)^{\frac{1}{2}}$$

Ce qui implique trivialement que  $\|\nabla u\|_{L^2(\Omega)} \leq \|u\|_{H^1(\Omega)}$  et  $\|\nabla v\|_{L^2(\Omega)} \leq \|v\|_{H^1(\Omega)}$ . De même on définit la forme linéaire suivante :

$$L(v) = \int_{\Omega} f v \, dx$$

Elle est continue :

$$|L(v)| = \left| \int_{\Omega} f v \, dx \right| \leq \|f\|_{L^2(\Omega)} \|v\|_{L^2(\Omega)}$$

Ainsi puisque  $\mathcal{D}(\Omega)$  est dense dans  $H_0^1(\Omega)$  alors : pour tout  $v \in H_0^1(\Omega)$ ,

$$\int_{\Omega} \nabla u \cdot \nabla v \, dx = \int_{\Omega} f v \, dx$$

Ce nouveau problème est équivalent à notre problème initiale. (La réciproque est immédiate)

## A.4 Démonstration du théorème du minimale de la fonctionnelle d'énergie

Dans ce paragraphe on montre que si  $a$  est de plus symétrique, alors la solution unique du problème précédent est aussi l'unique solution qui vérifie :

$$J(u) = \inf_{v \in H} J(v)$$

où  $J(v) = \frac{1}{2}a(v, v) - b(v)$

**Démonstration :**

— **Étape 1 : Si  $u$  est solution, alors  $u$  minimise  $J$**

Soit  $v \in V$ . On pose  $v = u + w$  avec  $w \in V$ . On a :

$$J(v) = J(u + w) = \frac{1}{2}a(u + w, u + w) - L(u + w) =$$

$$\frac{1}{2} [a(u, u) + a(u, w) + a(w, u) + a(w, w)] - L(u) - L(w)$$

Par symétrie de  $a$ ,  $a(u, w) = a(w, u)$ , on obtient :

$$J(v) = \left[ \frac{1}{2}a(u, u) - L(u) \right] + [a(u, w) - L(w)] + \frac{1}{2}a(w, w)$$

$$= J(u) + [a(u, w) - L(w)] + \frac{1}{2}a(w, w)$$

Comme  $u$  est solution du problème variationnel,  $a(u, w) = L(w)$ , donc le terme central s'annule. De plus, par coercivité,  $\frac{1}{2}a(w, w) \geq 0$ . On a donc bien  $J(v) \geq J(u)$  pour tout  $v \in V$ .

### Étape 2 : Si $u$ minimise $J$ , alors $u$ est solution

Soit  $v \in V$  et  $t \in \mathbb{R}$ . La fonction scalaire  $g(t) = J(u + tv)$  admet un minimum en  $t = 0$ , donc  $g'(0) = 0$ . On développe  $g(t)$  en utilisant la symétrie de  $a$  :

$$\begin{aligned} g(t) &= \frac{1}{2}a(u + tv, u + tv) - L(u + tv) \\ &= J(u) + t[a(u, v) - L(v)] + \frac{t^2}{2}a(v, v) \end{aligned}$$

En dérivant par rapport à  $t$ , on obtient :

$$g'(t) = [a(u, v) - L(v)] + ta(v, v)$$

En évaluant en  $t = 0$ ,  $g'(0) = a(u, v) - L(v) = 0$ . Ainsi,  $a(u, v) = L(v)$  pour tout  $v \in V$ .

## B Extraits essentiels du code source

Ce document présente l'implémentation des différentes architectures de réseaux de neurones et des algorithmes d'optimisation (PINN et DeepRitz) utilisés dans ce projet.

### B.1 Architecture et optimisation pour PINN

```

1 class snn(nn.Module):
2     def __init__(self, n_neurons):
3         super().__init__()
4         self.couche = nn.Linear(1, n_neurons)
5         self.out = nn.Linear(n_neurons, 1)
6
7     def forward(self, x):
8         x = activation_fct(self.couche(x))
9         x = self.out(x)
10        return x
11
12 def f_forward(x, net, params, buffers):

```

```

13     return torch.func.functional_call(net, (params, buffers), x.unsqueeze
    (-1)).squeeze()
14
15 # Gradients d'ordre 1 et 2 pour la fonction de perte PINN
16 f_forward_x = grad(f_forward, argnums=0)
17 f_forward_xx = grad(f_forward_x, argnums=0)

```

Listing 1 – Architecture du réseau SNN et calcul des gradients (autograd)

```

1 def optimal_snn(net: snn, silent=False):
2     epochs = 1000
3     N = 1000
4     losses = np.zeros(epochs)
5
6     optimizer = torch.optim.Adam(net.parameters(), lr=0.01)
7     params = dict(net.named_parameters())
8     buffers = dict(net.named_buffers())
9
10    grid = torch.linspace(0, 1, N)
11    solution_grid = vmap(solution, in_dims=(0,))(grid)
12    f_grid = vmap(f, in_dims=(0,))(grid)
13
14    for i in range(epochs):
15        loss = 0
16        f_forward_xx_grid = vmap(f_forward_xx, in_dims=(0, None, None, None)
17    )(grid, net, params, buffers)
18
19        bc_0 = torch.tensor(0.0)
20        bc_1 = torch.tensor(1.0)
21
22        # Calcul de la perte: quation diff rentielle + conditions aux
23    limites
24        loss += ((f_forward_xx_grid - f_grid)**2).mean()
25        loss += f_forward(bc_0, net, params, buffers)**2
26        loss += f_forward(bc_1, net, params, buffers)**2
27        losses[i] = loss.item()
28
29        optimizer.zero_grad()
30        loss.backward()
31        optimizer.step()

```

Listing 2 – Boucle d'entraînement Physics-Informed Neural Network (PINN)

## B.2 Architecture Deep Neural Network (DNN)

```

1 class NN(nn.Module):
2     def __init__(self, n_neurons, n_couches, activation_name='tanh'):
3         super().__init__()
4         self.couches = nn.ModuleList()

```

```

5         self.couches.append(nn.Linear(1, n_neurons))
6         self.activation = activation_functions_map[activation_name]
7
8         for i in range(n_couches):
9             self.couches.append(nn.Linear(n_neurons, n_neurons))
10
11        self.couches.append(nn.Linear(n_neurons, 1))
12
13    def forward(self, x):
14        x = self.couches[0](x)
15        for i in range(1, len(self.couches)):
16            x = self.activation(x)
17            x = self.couches[i](x)
18    return x

```

Listing 3 – Architecture du réseau profond utilisé par DeepRitz

### B.3 Méthode DeepRitz (Équation de Poisson)

```

1 def DeepRitz(net: NN, N_train, silent=False):
2     grid = torch.linspace(0, 1, N_train)
3     f_grid = vmap(f, in_dims=(0,))(grid)
4     params = dict(net.named_parameters())
5     buffers = dict(net.named_buffers())
6     optimizer = torch.optim.Adam(net.parameters(), lr=0.1)
7
8     bc_0 = torch.tensor([0.0])
9     bc_1 = torch.tensor([1.0])
10
11    for i in range(3000):
12        u_x_grid = vmap(f_forward_x, in_dims=(0, None, None, None))(grid.
squeeze(-1), net, params, buffers)
13        u_grid = vmap(f_forward, in_dims=(0, None, None, None))(grid.squeeze
(-1), net, params, buffers)
14
15        # Minimisation de la fonctionnelle J(u) et p nalisation aux bords
16        loss = 0.5 * (u_x_grid**2).mean() - (u_grid * f_grid).mean() + \
17            10 * f_forward(bc_0, net, params, buffers)**2 + \
18            10 * f_forward(bc_1, net, params, buffers)**2
19
20        optimizer.zero_grad()
21        loss.backward()
22        optimizer.step()

```

Listing 4 – Algorithme DeepRitz pour l'équation de Poisson

### B.4 Méthode DeepRitz (Deuxième équation)



```

1 def DeepRitz2(net, alpha, beta, N_train, silent=False):
2     grid = torch.linspace(0, 1, N_train)
3     f_grid = vmap(f, in_dims=(0,))(grid)
4     params = dict(net.named_parameters())
5     buffers = dict(net.named_buffers())
6     optimizer = torch.optim.Adam(net.parameters(), lr=0.01)
7
8     bc_0 = torch.tensor([0.0])
9     bc_1 = torch.tensor([1.0])
10
11     for i in range(1000):
12         u_x_grid = vmap(f_forward_x, in_dims=(0, None, None, None))(grid.
squeeze(-1), net, params, buffers)
13         u_grid = vmap(f_forward, in_dims=(0, None, None, None))(grid.squeeze
(-1), net, params, buffers)
14
15         # Nouvelle fonctionnelle J(u) incluant le terme d'ordre 0
16         loss = 0.5 * (u_x_grid**2).mean() + (u_grid * f_grid).mean() + 0.5 *
(u_grid**2).mean() + \
17             alpha * f_forward(bc_0, net, params, buffers)**2 + \
18             beta * f_forward(bc_1, net, params, buffers)**2
19
20         optimizer.zero_grad()
21         loss.backward()
22         optimizer.step()

```

Listing 5 – Algorithme DeepRitz adapté pour l'équation avec terme d'ordre 0

## Références

- [1] G. Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2(4) :303–314, December 1989.
- [2] Weinan E and Bing Yu. The deep ritz method : A deep learning-based numerical algorithm for solving variational problems. *Communications in Mathematics and Statistics*, 6(1) :1–12, February 2018.
- [3] M. Raissi, P. Perdikaris, and G.E. Karniadakis. Physics-informed neural networks : A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378 :686–707, February 2019.